

Code Clash 2026 — PRD & Technical Specification

1. Executive Summary

Problem: Pallavi Engineering College (Hyderabad) requires a fully-automated, proctored coding competition platform for ~100–200 concurrent student participants, with zero budget. The system must handle registration, dual-round exams (TDD + Debugging), sandboxed code execution, anti-cheat enforcement, and a live admin dashboard — all running within a single 2-hour window on **Friday, 7 August 2026** (Round 1 unlocks 13:45 IST, 30 min; Round 2 unlocks 14:30 IST, 30 min).

Business goals:

- G1: Two frictionless, bulletproof auth paths (Email/Password, Google OAuth) that always produce a populated `profiles` row.
- G2: Exam integrity via mandatory fullscreen, cumulative background-time limits, single-attempt enforcement, and per-session state machines.
- G3: Real-time operational visibility for admins (live registrations, active sessions, submissions, violations).
- G4: Automatic grading against hidden test cases with deterministic tie-breaking, published on a live leaderboard.

Scope: Complete build, already implemented and compiling. Remaining work: end-to-end verification against the live Supabase instance, Realtime publication confirmation, sample data seeding, and Vercel deployment.

2. System Architecture

Topology: Client → Next.js 16 App Router (Vercel Edge/Node) → Supabase (PostgreSQL + Auth + Realtime) and Piston API (code execution). No self-hosted backend; all business logic lives in Next.js route handlers and Supabase database functions.

Component boundaries:

- `src/lib/supabase/` — three client factories: `client.ts` (browser), `server.ts` (route handlers), `middleware.ts` (proxy/edge session refresh + route guards). `src/proxy.ts` is the Next.js 16 entry (Turbopack builds `proxy` in place of legacy `middleware.ts`) delegating to `updateSession()`.
- **Route guard layer** (`proxy.ts`) — enforces: unauthenticated → `/login`; authenticated on `/login` `/signup` → role-based redirect; student on `/admin` → `/dashboard`; student with `roll_no` / `branch` = `'EMPTY'` on any `/dashboard` route (except `/dashboard/onboarding`) → `/dashboard/onboarding`.
- **API route handlers** (`/api/*`) — thin HTTP layer over Supabase with server-side validation: `exam-sessions` (state machine), `submissions` (Piston evaluation loop),

`violations` `questions` `settings` `leaderboard` `execute`

- **Client exam shell** (`/dashboard/exam/[examId]/live`) — Monaco editor + Markdown problem panel + anti-cheat hook + countdown timer; single-attempt gate on mount.
- **Admin app** (`/admin/*`) — CRUD, leaderboard with dual-tab ranking, live Realtime counters.

Data flow (exam lifecycle): Student starts exam → `POST /api/exam-sessions` creates `in_progress` session (or returns existing; blocks if `completed`) → student codes, runs sample tests via `POST /api/execute` (Piston) → submit/auto-submit → per-question evaluation loop over all test cases (sample + hidden) via Piston → `submissions` rows upserted → session PATCHed to `completed` → leaderboard recomputed → admin dashboard updates via Realtime.

3. Tech Stack & Dependencies

Layer	Choice	Notes
Framework	Next.js 16.2.12 (App Router, Turbopack)	Route handlers use <code>Promise</code> based <code>params</code> <code>proxy.ts</code> replaces <code>middleware.ts</code>
Language	TypeScript (strict)	
Styling	Tailwind CSS (dark theme)	Tokens: <code>bg-background</code> <code>bg-surface</code> <code>border-border</code> <code>primary-light</code> <code>success</code> <code>danger</code> <code>warning</code> <code>accent</code>
Editor	Monaco Editor (<code>@monaco-editor/react</code>)	Read <code>AGENTS.md</code> note: Next 16 docs in <code>node_modules/next/dist/docs/</code>
Backend	Supabase (PostgreSQL + Auth + Realtime)	Anon key client-side; RLS is the security boundary
Execution	Piston API v2 (<code>https://emkc.org/api/v2/piston</code>)	Public, no key. Env <code>NEXT_PUBLIC_PISTON_URL</code> overrides; self-hostable via Docker
Markdown	<code>react-markdown</code> + <code>remark-gfm</code>	
Icons	<code>lucide-react</code>	
Deployment	Vercel	Free tier

Supported contest languages: Python, Java, C, C++ (DSA-only problems).

4. Codebase Directory Map

```

code-clash-2026/
├─ proxy.ts # Next 16 proxy entry (auth
guard + onboarding guard)
├─ supabase/migrations/
│   └─ 20260804_fix_handle_new_user_google_oauth.sql # Trigger:
Google-safe profile insert
│   └─ 20260804_fix_auth_profiles_complete.sql # year column,
RLS, dual-path trigger (EXECUTED)
└─ src/
    ├─ lib/supabase/{client,server,middleware}.ts
    ├─ types/database.ts # Typed Database schema
(all 8 tables) + Piston types
    ├─ hooks/
    │   └─ useAuth.tsx # AuthProvider: session +
profile context
    │   └─ useAntiCheat.ts # 15s cumulative bg timer,
fullscreen, key blockers
    │   └─ useExamTimer.ts # Server-time countdown,
auto-submit on expiry
    │   └─ components/auth/
    │       └─ LoginForm.tsx # Email/password sign-in,
inline errors
    │   └─ SignupForm.tsx # Full details + year;
email-verification success state
    │   └─ GoogleOAuthButton.tsx # signInWithOAuth →
/auth/callback
    │   └─ components/exam/ # CodeEditor, ProblemPanel,
TestResults, Timer, ViolationModal
    │   └─ app/
    │       └─ (auth)/{login,signup}/ # Auth pages under shared
PEC-branded layout
    │   └─ auth/callback/route.ts # OAuth code exchange +
error passthrough to /login?error=
    │   └─ dashboard/onboarding/page.tsx # Post-Google mandatory
details form
    │   └─ dashboard/exam/[examId]/live/page.tsx # Exam shell
(TDD/DEBUGGING aware)
    │   └─ admin/{page,questions,settings,leaderboard}/page.tsx

```

```
| └─ api/{execute,settings,questions,questions/[id],exam-  
sessions,submissions,violations,leaderboard}/route.ts
```

5. Core Logic & Business Rules

5.1 profiles schema

```
id uuid PK REFERENCES auth.users(id), email text, full_name text,  
roll_no text, branch text, year text (default '1'), role text  
( 'student'| 'admin', default 'student'), is_active bool,  
created_at/updated_at timestampz
```

5.2 Trigger `handle_new_user` (AFTER INSERT ON `auth.users`, SECURITY DEFINER)

- Reads `NEW.raw_user_meta_data`
- **Email path:** `roll_no` / `branch` / `year` present in metadata → inserted directly.
- **Google path:** metadata lacks them → `'EMPTY'` `'EMPTY'` `'1'` hardcoded; name resolved from `full_name` → `name` → `preferred_name` → email-prefix.
- `ON CONFLICT (id) DO UPDATE` with guarded CASE clauses (never overwrite a filled field with `'EMPTY'`); `EXCEPTION WHEN OTHERS` swallows failure (logs warning) so `auth.user` creation is never blocked.

5.3 RLS policies (profiles)

```
ENABLE ROW LEVEL SECURITY;  
SELECT → USING (auth.uid() = id)           -- TO  
authenticated  
UPDATE → USING (auth.uid() = id) WITH CHECK (auth.uid() = id)  
ALL    → TO service_role, USING (true)      -- admin API  
routes
```

This resolves the historical `Database error saving new user` and onboarding `Failed to save` failures.

5.4 Auth routing decision tree

1. `signup()` → if `data.user && !data.session` → email confirmation required → render "Check your email" success state (no redirect). If session returned → `/dashboard`.
2. Google OAuth → `redirectTo: /auth/callback` → `exchangeCodeForSession` → callback inspects `error` / `error_description` params first (redirects `/login?error=<msg>`, rendered as toast-style banner via `useSearchParams` in Suspense); missing profile after exchange → `/dashboard/onboarding`.
3. Middleware: any `/dashboard` hit with `roll_no === 'EMPTY'` or `branch === 'EMPTY'` → hard redirect to onboarding (server-side, unbypassable).

5.5 Exam session state machine (server-enforced in `/api/exam-sessions`)

pending → in_progress → completed (no reverse transitions; 400 on violation)

POST is idempotent resume; `completed` sessions block re-entry (live page shows "already submitted" screen).

5.6 Anti-cheat (client hook + server `violations` table)

Cumulative **15,000 ms** background accumulation via `visibilitychange` + 250 ms interval (paused on return, never reset); warning at ≤5 s remaining, auto-submit at 0; fullscreen-exit events and blocked shortcuts (`F12`, Ctrl/Cmd+C/V/X, Ctrl+Shift+I/J, Ctrl+U, right-click) recorded as `violations` rows. Gate: `settings.is_anti_cheat_enabled`.

5.7 Grading/leaderboard

Submissions evaluated at submit time against every test case (sample + hidden) through Piston; normalized output comparison; ranking per round = hidden tests passed → execution time → time taken.

6. Current Status & Pending Tasks

Complete & verified compiling (`npm run build` passes, 20 routes + proxy): auth UI/routing (both paths), onboarding guard, trigger + RLS migrations (user executed in Supabase), email-verification success state, live exam shell, Piston wrapper, leaderboard.

Hardening features — implemented in code, pending E2E verification + deployment:

Feature	Status	Location
Single-attempt enforcement (session	Implemented	<code>src/app/api/exam-sessions/route.ts</code> live page <code>status === "completed"</code> gate

state machine, re-entry block)		
15 s cumulative anti-cheat timer (background accumulation)	Implemented	<code>src/hooks/useAntiCheat.ts</code> <code>maxBackgroundTimeMs = 15000</code>
Real-time admin dashboard (Realtime counters on 4 tables)	Implemented;	<code>src/app/admin/page.tsx</code> <code>channel("admin-realtime")</code>
Distinct TDD vs. DEBUGGING UIs (question_type selector, badges, buggy starter code)	Implemented	<code>src/app/admin/questions/page.tsx</code> live page, <code>ProblemPanel</code>

Remaining tasks for the next engineer:

1. Confirm Realtime publications on `profiles`, `exam_sessions`, `submissions`, `violations` (SQL migration does not cover this — dashboard toggles required).
2. Seed sample questions/test cases via admin UI; dry-run both rounds with test accounts.
3. Verify Google OAuth in production (authorized redirect URIs in Google Cloud Console + Supabase).
4. Test auto-submit path (timer expiry and anti-cheat trigger) against hidden tests.
5. Deploy to Vercel; sanity-check `.env.local` → Vercel env vars
(`NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`, `NEXT_PUBLIC_PISTON_URL`).